

Phase 6: Quick Sort Recursion and Complexity — Full Structured Beginner Note

1. Current Progress Before Phase 6

Before Phase 6, you have already learned five important phases.

Phase 1: Sorting Foundations

You learned the basic language used to understand sorting algorithms:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already sorted or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(n)$	Work grows directly with input size
$O(n^2)$	Work grows very quickly, often because of nested loops
$O(n \log n)$	Usually a fast class for comparison sorting
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, "Does it sort?"

A better question is:

How does it sort?
How much time does it take?
How much memory does it use?
Is it stable?
Is it adaptive?

Phase 2: Bubble Sort

Bubble Sort compares neighboring values and swaps them if they are in the wrong order.

Example:

8 5

Since:

8 > 5

Bubble Sort swaps them:

5 8

Important points from Bubble Sort:

- It compares adjacent elements.
- It uses swaps.
- One full scan is called a pass.
- After each pass, one large value is fixed at the end.
- After k passes, the k largest values are fixed at the end.
- It can be adaptive if we use a `flag`.
- It is stable if we use `>` instead of `>=`.
- Best-case time is $O(n)$ with the adaptive flag.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

Phase 3: Insertion Sort

Insertion Sort builds a sorted left part.

Example:

```
[5 8] 7 3 2
```

The left part `[5 8]` is sorted.

Now we insert `7` into the correct position:

```
[5 7 8] 3 2
```

Important points from Insertion Sort:

- It inserts one value into a sorted left part.
 - It starts from index `1` because the first element alone is already sorted.
 - It stores the selected value in `x`.
 - It shifts bigger values to the right.
 - It is naturally adaptive.
 - It is stable if we use `>` instead of `>=`.
 - Best-case time is $O(n)$.
 - Worst-case time is $O(n^2)$.
 - Space complexity is $O(1)$.
-

Phase 4: Selection Sort

Selection Sort selects the smallest value from the unsorted part and puts it at the front.

Example:

```
8 6 3 2 5 4
```

The smallest value is `2`, so Selection Sort swaps it with the first value:

```
2 6 3 8 5 4
```

Important points from Selection Sort:

- It selects the minimum value.
- It fills the array from left to right.
- It uses `i`, `j`, and `k`.
- `i` is the position being filled.
- `j` scans the unsorted part.

- `k` stores the index of the smallest value found so far.
- It swaps only after the inner scan finishes.
- It performs fewer swaps than Bubble Sort.
- It is not adaptive.
- It is usually not stable.
- Best-case time is $O(n^2)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- After `k` passes, the `k` smallest values are fixed at the front.

Phase 5: Quick Sort Partitioning

Phase 5 introduced the heart of Quick Sort: **partitioning**.

Partitioning means:

Choose one value, called the pivot, and place it in its final sorted position.

After partitioning:

smaller values | pivot | greater values

Example:

50 70 60 90 40 80 10 20 30 ∞

The pivot is `50`.

After partitioning:

40 30 20 10 | 50 | 80 90 60 70

Now `50` is in its final sorted position.

But the left side is not fully sorted:

40 30 20 10

And the right side is not fully sorted:

80 90 60 70

So partitioning alone is not enough.

This is why Phase 6 is needed.

2. Why Phase 6 Comes After Phase 5

Quick Sort has two big ideas:

1. **Partitioning**
2. **Recursion**

Phase 5 taught partitioning.

Phase 6 teaches recursion.

In Phase 5, we learned how to fix one pivot.

In Phase 6, we learn how to use the same partitioning idea again and again until the whole array becomes sorted.

Simple idea:

Partition the array.
Then Quick Sort the left side.
Then Quick Sort the right side.

Memory trick:

Partition, sort left, sort right.

3. Main Goal of Phase 6

The main goal of Phase 6 is to understand full Quick Sort.

In simple words:

Quick Sort sorts an array by repeatedly partitioning smaller and smaller parts.

Phase 6 covers:

- Recursive Quick Sort
- How the `QuickSort()` function calls itself
- The base case
- Small-list behavior
- Best-case time complexity
- Average-case time complexity
- Worst-case time complexity
- Why already sorted data can be bad for first-element pivot Quick Sort
- Why reverse sorted data can also be bad
- How better pivot choices can improve Quick Sort
- Quick Sort vs Selection Sort
- Full C implementation
- Common beginner mistakes

4. What Is Recursion?

Recursion means:

A function calls itself.

In Quick Sort, the function `QuickSort()` calls itself on smaller parts of the array.

Example:

```
QuickSort(whole array)
    QuickSort(left side)
    QuickSort(right side)
```

Then the left side may also be divided:

```
QuickSort(left side)
    QuickSort(left-left side)
    QuickSort(left-right side)
```

And the right side may also be divided:

```
QuickSort(right side)
  QuickSort(right-left side)
  QuickSort(right-right side)
```

This continues until the parts become too small to sort.

5. Why Quick Sort Uses Recursion

After one partition, only the pivot is fixed.

Example:

```
40 30 20 10 | 50 | 80 90 60 70
```

The pivot `50` is fixed.

But the left side still needs sorting:

```
40 30 20 10
```

The right side also still needs sorting:

```
80 90 60 70
```

So Quick Sort says:

```
Use Quick Sort again on the left side.
Use Quick Sort again on the right side.
```

That is recursion.

The same function solves smaller versions of the same problem.

6. Divide and Conquer Idea

Quick Sort is a **divide and conquer** algorithm.

That means:

Divide the big problem into smaller problems.
Solve the smaller problems.
Combine the result naturally.

For Quick Sort:

Divide

Partition the array around a pivot.

smaller values | pivot | greater values

Conquer

Recursively sort the left side and right side.

Combine

There is no special merge step like Merge Sort.

Why?

Because after partitioning:

left side values are smaller than pivot
right side values are greater than pivot

So once the left and right sides are sorted, the whole array is sorted automatically.

7. What Is the Base Case?

A base case is the condition that stops recursion.

Without a base case, recursion would continue forever.

Quick Sort stops when the current section has zero or one element.

Why?

Because:

A one-element list is already sorted.
A zero-element list is also finished.

In code, the base case is handled by:

```
if (low < high)
```

This means:

Only do partitioning if the section has enough elements to sort.

If `low` is not less than `high`, Quick Sort does nothing for that section.

8. Small-List Behavior

Quick Sort does useful work only when there are at least two elements in the section.

Case 1: One element

```
[30]
```

This is already sorted.

No partition is needed.

Case 2: Zero elements

```
[]
```

There is nothing to sort.

No partition is needed.

Case 3: Two or more elements

```
[40 10]
```

Now Quick Sort can partition the section and sort it.

So the rule is:

If the section has fewer than two elements, stop.

9. Important Variables in Phase 6

Variable	Simple meaning
<code>A[]</code>	The array being sorted
<code>low</code>	Starting index of the current section
<code>high</code>	Ending boundary index in this partition version
<code>pivot</code>	The selected value that will be placed correctly
<code>i</code>	Moves from left to right in partitioning
<code>j</code>	Moves from right to left in partitioning
<code>j</code> returned by <code>Partition()</code>	Final position of the pivot
<code>INT32_MAX</code>	Infinity marker used as a boundary in this code
<code>QuickSort()</code>	Recursive sorting function
<code>Partition()</code>	Function that places one pivot correctly

10. Full Quick Sort Logic

Quick Sort works like this:

1. Choose a pivot.
2. Partition the array around the pivot.
3. The pivot reaches its final sorted position.
4. Recursively sort the left side.
5. Recursively sort the right side.

Pseudocode:

```
QuickSort(A, low, high):  
    if low < high:  
        j = Partition(A, low, high)
```

```
QuickSort(A, low, j)
QuickSort(A, j + 1, high)
```

Here:

- `Partition(A, low, high)` places one pivot correctly.
- `j` is the final pivot position.
- `QuickSort(A, low, j)` sorts the left side.
- `QuickSort(A, j + 1, high)` sorts the right side.

11. Why This Code Uses `QuickSort(A, low, j)`

In many books, you may see this:

```
QuickSort(A, low, j - 1);
QuickSort(A, j + 1, high);
```

But in this curriculum's partition method, the recursive calls are:

```
QuickSort(A, low, j);
QuickSort(A, j + 1, high);
```

This is important.

Different partition methods use different boundaries.

So do not mix code from different textbooks unless you understand the partition method.

For this exact partition method, use:

```
QuickSort(A, low, j);
QuickSort(A, j + 1, high);
```

12. Full Phase 6 Program in C

This program uses the same partition method from Phase 5.

It also uses `INT32_MAX` as the infinity marker.

```

#include <stdio.h>
#include <stdint.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int Partition(int A[], int low, int high) {
    int pivot = A[low];
    int i = low;
    int j = high;

    do {
        do {
            i++;
        } while (A[i] <= pivot);

        do {
            j--;
        } while (A[j] > pivot);

        if (i < j) {
            swap(&A[i], &A[j]);
        }
    } while (i < j);

    swap(&A[low], &A[j]);
    return j;
}

void QuickSort(int A[], int low, int high) {
    int j;

    if (low < high) {
        j = Partition(A, low, high);
        QuickSort(A, low, j);
        QuickSort(A, j + 1, high);
    }
}

int main() {
    int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};
    int high = 9; // index of infinity marker

    QuickSort(A, 0, high);
}

```

```

    for (int i = 0; i < high; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}

```

Expected output:

```
10 20 30 40 50 60 70 80 90
```

13. Explanation of the Complete Program

The program has four main parts:

1. `swap()` function
2. `Partition()` function
3. `QuickSort()` function
4. `main()` function

14. The `swap()` Function

```

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

```

This function swaps two integer values.

It uses pointers because it must change the real values inside the array.

Code	Meaning
<code>int *x</code>	<code>x</code> is a pointer to an integer
<code>int *y</code>	<code>y</code> is a pointer to an integer
<code>*x</code>	Value stored at the address inside <code>x</code>

Code	Meaning
<code>*y</code>	Value stored at the address inside <code>y</code>
<code>temp</code>	Temporary variable used to protect one value

When the program calls:

```
swap(&A[i], &A[j]);
```

The `&` means:

address of

So the function receives the addresses of the array elements and swaps the real values.

15. The `Partition()` Function

```
int Partition(int A[], int low, int high)
```

This function places one pivot in its final sorted position.

It returns the final index of the pivot.

Choosing the pivot

```
int pivot = A[low];
```

This version chooses the first element of the current section as the pivot.

Example:

50 70 60 90 40 80 10 20 30 ∞

If `low = 0`, then:

pivot = 50

Starting `i` and `j`

```
int i = low;  
int j = high;
```

- `i` starts from the left side.
- `j` starts from the right boundary.
- In this code, `high` points to the infinity marker.

Moving `i`

```
do {  
    i++;  
} while (A[i] <= pivot);
```

This moves `i` to the right.

It stops when it finds a value greater than the pivot.

Simple meaning:

Move right until you find something too big for the left side.

Moving `j`

```
do {  
    j--;  
} while (A[j] > pivot);
```

This moves `j` to the left.

It stops when it finds a value less than or equal to the pivot.

Simple meaning:

Move left until you find something small enough for the left side.

Swapping wrong-side values

```
if (i < j) {  
    swap(&A[i], &A[j]);  
}
```

If `i` is still before `j`, the two stopped values are on the wrong sides.

So they are swapped.

Final pivot swap

```
swap(&A[low], &A[j]);
```

After `i` and `j` cross, `j` gives the final position of the pivot.

So the pivot is swapped with `A[j]`.

Return `j`

```
return j;
```

The function returns `j` because `j` is the pivot's final position.

16. The `QuickSort()` Function

```
void QuickSort(int A[], int low, int high) {  
    int j;  
  
    if (low < high) {  
        j = Partition(A, low, high);  
        QuickSort(A, low, j);  
        QuickSort(A, j + 1, high);  
    }  
}
```

This is the main recursive function.

Function header

```
void QuickSort(int A[], int low, int high)
```

Parameter	Meaning
A[]	The array to sort
low	Starting index of the current section
high	Ending boundary index for this version

The base case condition

```
if (low < high)
```

This prevents endless recursion.

It means:

```
Only sort if the section is large enough.
```

Partition call

```
j = Partition(A, low, high);
```

This places one pivot in its final position.

The final pivot index is stored in `j`.

Left recursive call

```
QuickSort(A, low, j);
```

This sorts the left side.

Right recursive call

```
QuickSort(A, j + 1, high);
```

This sorts the right side.

17. The `main()` Function

```
int main() {
    int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};
    int high = 9; // index of infinity marker

    QuickSort(A, 0, high);

    for (int i = 0; i < high; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}
```

This function:

1. Creates the array.
2. Adds `INT32_MAX` as the infinity marker.
3. Sets `high = 9`, the index of the infinity marker.
4. Calls `QuickSort(A, 0, high)`.
5. Prints only the real values.
6. Does not print the infinity marker.

The print loop is:

```
for (int i = 0; i < high; i++)
```

It prints indices `0` to `8`.

It does not print index `9`, because index `9` contains `INT32_MAX`.

18. Why the Infinity Marker Is Used

The array is:

```
50 70 60 90 40 80 10 20 30 ∞
```

The infinity marker is represented in C as:

INT32_MAX

This is a very large integer.

It acts like a wall at the end of the array.

Why is it needed?

Because `i` moves right using this code:

```
do {  
    i++;  
} while (A[i] <= pivot);
```

If the pivot is very large, `i` may keep moving right.

The infinity marker gives `i` a safe place to stop.

Simple meaning:

The infinity marker prevents `i` from running outside the array.

Important:

INT32_MAX is not real data.
It is only a boundary marker.

19. Beginner Dry Run of Full Quick Sort

Start with:

50 70 60 90 40 80 10 20 30 ∞

First partition

The pivot is:

50

After partitioning:

40 30 20 10 | 50 | 80 90 60 70

Now 50 is fixed.

Quick Sort now sorts:

Left side: 40 30 20 10
Right side: 80 90 60 70

Sorting the left side

Left side:

40 30 20 10

Pivot is:

40

After partitioning this section, 40 moves to its correct position inside the left side.

The smaller values stay to its left.

Now Quick Sort continues on even smaller sections.

Sorting the right side

Right side:

80 90 60 70

Pivot is:

80

After partitioning this section, 80 moves to its correct position inside the right side.

Values smaller than 80 go to the left of it.

Values greater than 80 go to the right of it.

Recursion continues

Quick Sort keeps doing this:

```
Partition current section.  
Sort left part.  
Sort right part.
```

Eventually, all sections become one-element or zero-element sections.

At that point, recursion stops.

Final sorted array:

10 20 30 40 50 60 70 80 90

20. Recursion Tree Idea

Quick Sort can be imagined like a tree.

At the top, we sort the full array:

```
QuickSort(whole array)
```

Then it splits into two calls:

```
QuickSort(left side)    QuickSort(right side)
```

Then each side splits again:

```
QuickSort(left-left) QuickSort(left-right) QuickSort(right-left)
QuickSort(right-right)
```

The process continues until the parts are too small.

This tree idea helps us understand time complexity.

21. Why Quick Sort Can Be Fast

One partition step scans the current section.

If the section has n elements, one partition takes:

$O(n)$

If the pivot divides the array into two balanced halves, the work looks like this:

```
Level 1: n work
Level 2: n work total
Level 3: n work total
Level 4: n work total
...
```

Each level does about n total work.

The number of levels is about:

$\log n$

So the total work is:

$n \times \log n$

Therefore, Quick Sort can run in:

$O(n \log n)$

This is much better than $O(n^2)$ for large arrays.

22. Best-Case Time Complexity

The best case happens when the pivot divides the array into two almost equal halves each time.

Example idea:

```
left half | pivot | right half
```

The two sides are nearly the same size.

This creates a balanced recursion tree.

Best-case time:

```
O(n log n)
```

Why?

- Each level processes about `n` elements.
- There are about `log n` levels.
- Total work becomes `n log n`.

23. Average-Case Time Complexity

The average case happens when the pivots are not always perfect, but they are usually not terrible.

In random-looking data, partitions are often reasonably balanced.

Average-case time:

```
O(n log n)
```

This is one reason Quick Sort is popular.

In practice, Quick Sort is often very fast for average input.

24. Worst-Case Time Complexity

The worst case happens when the pivot is always the smallest or largest value.

Then the partition is extremely unbalanced.

Instead of this:

```
half | pivot | half
```

We get this:

```
empty side | pivot | almost full side
```

That is bad because the problem does not shrink evenly.

Worst-case time:

```
 $O(n^2)$ 
```

25. Worst Case on Already Sorted Data

Suppose the array is already sorted:

```
10 20 30 40 50
```

If the first element is always chosen as pivot, then the pivot is always the smallest value.

First pivot:

```
10
```

The split becomes:

```
empty left side | 10 | 20 30 40 50
```

Next pivot:

20

Again:

empty left side | 20 | 30 40 50

This keeps happening.

So the work becomes like:

$$n + (n - 1) + (n - 2) + \dots + 1$$

That grows like:

$$O(n^2)$$

Important beginner point:

Already sorted input can be bad for first-element pivot Quick Sort.

26. Worst Case on Reverse Sorted Data

Reverse sorted data can also be bad.

Example:

50 40 30 20 10

If the first element is chosen as pivot, the pivot may be the largest value.

The split can become very unbalanced again:

almost full side | pivot | empty side

So reverse sorted input can also lead to:

$O(n^2)$

27. Complexity Summary Table

Case	What happens	Time complexity
Best case	Pivot divides the array into two almost equal halves	$O(n \log n)$
Average case	Pivot gives reasonably balanced parts most of the time	$O(n \log n)$
Worst case	Pivot is always smallest or largest	$O(n^2)$

28. Space Complexity of Quick Sort

Quick Sort does not need a large extra array.

It sorts by swapping values inside the original array.

So the partitioning part is in-place.

But Quick Sort uses recursion.

Recursion uses stack memory.

Balanced case stack space

If partitions are balanced, recursion depth is about:

$\log n$

So stack space is:

$O(\log n)$

Worst case stack space

If partitions are very unbalanced, recursion depth can become:

n

So worst-case stack space can become:

$O(n)$

Beginner summary:

Quick Sort is in-place for array movement, but recursion still uses stack space.

29. Is Quick Sort In-Place?

Usually, yes.

Quick Sort is usually called in-place because it rearranges values inside the original array.

It does not create another large array like Merge Sort.

However, it still uses recursion stack space.

So a careful statement is:

Quick Sort is in-place in terms of array storage, but it uses recursion stack memory.

30. Is Quick Sort Stable?

Usually, no.

Quick Sort often uses swaps that can move equal values around.

A stable sorting algorithm keeps equal values in their original relative order.

Quick Sort's common in-place form does not guarantee that.

So:

Common in-place Quick Sort is not stable.

31. Is Quick Sort Adaptive?

Not in this first-element pivot version.

An adaptive algorithm becomes faster when data is already sorted or nearly sorted.

But first-element pivot Quick Sort can become worse on already sorted data.

So this version is not adaptive in the same way as Bubble Sort with a flag or Insertion Sort.

Important idea:

Already sorted data is easy for Insertion Sort, but it can be bad for first-element pivot Quick Sort.

32. Pivot Improvements

The pivot choice is very important.

Bad pivot choices can cause $O(n^2)$ time.

Better pivot choices can reduce the chance of bad partitions.

Improvement 1: Middle Pivot

Instead of choosing the first element, choose the middle element.

This can help when the data is already sorted.

Example:

10 20 30 40 50

If we choose the first element, pivot is 10, which gives a bad split.

If we choose the middle element, pivot is 30, which gives a better split:

```
10 20 | 30 | 40 50
```

This is more balanced.

Improvement 2: Random Pivot

Randomized Quick Sort chooses a random pivot.

This makes it harder for the input to repeatedly produce bad pivots.

Important:

```
Random pivot does not make the worst case impossible.  
It only makes the worst case much less likely.
```

33. Why Pivot Choice Matters

Quick Sort is fast when partitions are balanced.

Balanced partition:

```
almost half | pivot | almost half
```

This gives:

```
 $O(n \log n)$ 
```

Unbalanced partition:

```
empty | pivot | almost everything
```

This gives:

```
 $O(n^2)$ 
```

So the quality of the pivot controls the shape of the recursion tree.

Good pivot means a short, balanced tree.

Bad pivot means a long, chain-like tree.

34. Quick Sort vs Selection Sort

Quick Sort and Selection Sort are related in an interesting way.

Both care about putting values into correct positions.

But they think differently.

Selection Sort thinking

Selection Sort chooses a position first.

Then it asks:

Which value belongs in this position?

Example:

8 6 3 2 5 4

For index , Selection Sort finds the smallest value, , and puts it there.

So Selection Sort is like:

Choose position → find correct value

Quick Sort thinking

Quick Sort chooses a value first.

That value is the pivot.

Then it asks:

Where does this pivot belong?

Example:

50 70 60 90 40 80 10 20 30

Quick Sort chooses pivot 50 and finds the correct position for 50.

So Quick Sort is like:

Choose value → find correct position

35. Quick Sort vs Selection Sort Table

Feature	Selection Sort	Quick Sort
Main idea	Choose a position and find the correct value	Choose a pivot value and find its correct position
First question	Which value belongs here?	Where does this pivot belong?
Main operation	Find minimum value	Partition around pivot
Recursion	No	Yes
Best case	$O(n^2)$	$O(n \log n)$
Average case	$O(n^2)$	$O(n \log n)$
Worst case	$O(n^2)$	$O(n^2)$
Extra array	No	No large extra array
Stability	Usually not stable	Usually not stable
Speed on large average data	Usually slow	Usually fast

36. Why Quick Sort Is Usually Faster Than Selection Sort

Selection Sort reduces swaps, but it still scans again and again to find the next smallest value.

Its time is usually:

$$O(n^2)$$

Quick Sort partitions the array into smaller parts.

If the parts are balanced, the work becomes:

$$O(n \log n)$$

This is why Quick Sort is usually much faster for large average data.

37. Quick Sort Properties

Property	Quick Sort result
Sorting type	Comparison-based sorting
Main idea	Partition around a pivot
Strategy	Divide and conquer
Uses recursion?	Yes
Best-case time	$O(n \log n)$
Average-case time	$O(n \log n)$
Worst-case time	$O(n^2)$
In-place?	Yes, for array movement
Extra array needed?	No large extra array
Recursion stack space	$O(\log n)$ balanced, $O(n)$ worst case
Stable?	Usually no
Adaptive?	Not in the first-element pivot version
Bad input for first pivot	Already sorted or reverse sorted data
Pivot improvement	Middle pivot or random pivot

38. Full Step-by-Step Memory Trick

Remember Quick Sort like this:

Choose pivot.
Partition around pivot.
Pivot becomes fixed.
Quick Sort left side.
Quick Sort right side.
Stop when section has zero or one element.

Shorter memory trick:

Partition, sort left, sort right.

Even shorter:

Pivot, divide, recurse.

39. Common Beginner Mistakes

Mistake 1: Thinking one partition sorts the whole array

Wrong idea:

After partitioning, the array is sorted.

Correct idea:

After partitioning, only the pivot is fixed.

Example:

40 30 20 10 | 50 | 80 90 60 70

Only 50 is fixed.

The left and right sides still need sorting.

Mistake 2: Forgetting the base case

Wrong idea:

```
Keep calling QuickSort forever.
```

Correct idea:

```
if (low < high)
```

The base case stops recursion when the section has zero or one element.

Mistake 3: Mixing partition methods

This curriculum uses:

```
QuickSort(A, low, j);  
QuickSort(A, j + 1, high);
```

Do not blindly replace it with:

```
QuickSort(A, low, j - 1);  
QuickSort(A, j + 1, high);
```

That may be correct for a different partition method, but not something to copy blindly here.

Mistake 4: Forgetting the infinity marker

This exact partition code uses:

```
INT32_MAX
```

at the end of the array.

If you forget it, `i` may move outside the array.

That can cause wrong behavior or a crash.

Mistake 5: Printing the infinity marker

The infinity marker is not real data.

Wrong:

```
for (int i = 0; i <= high; i++)
```

This may print `INT32_MAX`.

Better in this program:

```
for (int i = 0; i < high; i++)
```

This prints only the real values.

Mistake 6: Thinking sorted input is always easy

Sorted input is easy for Insertion Sort.

But sorted input can be bad for first-element pivot Quick Sort.

Example:

```
10 20 30 40 50
```

If the pivot is always the first element, the pivot is always the smallest.

That creates unbalanced partitions.

Mistake 7: Thinking random pivot removes the worst case completely

Random pivot reduces the chance of the worst case.

But it does not make the theoretical worst case impossible.

Correct idea:

```
Random pivot makes repeated bad pivot choices unlikely, not impossible.
```

40. Key Vocabulary for Phase 6

Term	Simple meaning
Quick Sort	Sorting algorithm based on partitioning and recursion
Pivot	Value chosen to be placed in its final position
Partition	Rearranging values around the pivot
Recursion	A function calling itself
Base case	Condition that stops recursion
Divide and conquer	Solving a problem by dividing it into smaller problems
Left side	Values before the pivot after partitioning
Right side	Values after the pivot after partitioning
<code>low</code>	Start index of the current section
<code>high</code>	End boundary index of the current section
<code>j</code>	Final pivot position returned by <code>Partition()</code>
Infinity marker	Boundary value used to stop scanning safely
<code>INT32_MAX</code>	Very large integer used as infinity marker
Balanced partition	Pivot splits array into two similar-sized parts
Unbalanced partition	Pivot creates one tiny side and one large side
Best case	Pivot keeps splitting the array evenly
Worst case	Pivot is always smallest or largest
Recursion stack	Memory used by recursive function calls

41. Quick Sort Evaluation Checklist

When studying Quick Sort, ask these questions:

Comparisons

Does Quick Sort compare values?

Yes.

It compares values with the pivot.

Movement

Does Quick Sort move values?

Yes.

It swaps values during partitioning.

Pivot behavior

Is the pivot splitting the array well?

If yes, Quick Sort is fast.

If no, Quick Sort can become slow.

Recursion

Does Quick Sort use recursion?

Yes.

It recursively sorts the left and right sides.

Base case

When does recursion stop?

It stops when the section has zero or one element.

Time complexity

What is the best case?

$O(n \log n)$

What is the average case?

$O(n \log n)$

What is the worst case?

$O(n^2)$

Memory

Does it need a large extra array?

No.

Does it use recursion stack memory?

Yes.

Stability

Is common in-place Quick Sort stable?

No.

Adaptiveness

Is first-element pivot Quick Sort adaptive for sorted data?

No.

Sorted data can actually be bad for it.

42. Final Beginner Summary

Phase 6 completes Quick Sort.

Phase 5 taught:

Put one pivot in the correct position.

Phase 6 adds:

Use recursion to do the same thing on the left and right sides.

The full Quick Sort idea is:

Partition the array.
The pivot becomes fixed.
Quick Sort the left side.
Quick Sort the right side.
Stop when the section has zero or one element.

Quick Sort is powerful because balanced partitions give:

$O(n \log n)$

But Quick Sort can become slow if pivot choices are bad, especially when the first element is used as pivot on already sorted or reverse sorted data.

Worst case:

$O(n^2)$

Final memory sentence:

Quick Sort = pivot + partition + recursion.